

ПРОЕКТИРОВАНИЕ И РЕАЛИЗАЦИЯ КОМПАКТНОГО МНОГОСЛОЙНОГО ПЕРЦЕПТРОНА НА ЯЗЫКЕ C++ ДЛЯ ЗАДАЧ КЛАССИФИКАЦИИ В УСЛОВИЯХ АППАРАТНЫХ ОГРАНИЧЕНИЙ БОРТОВЫХ СИСТЕМ КОСМИЧЕСКИХ АППАРАТОВ

Аннотация. В статье рассматривается подход реализации многослойного перцептрона на компилируемом языке программирования C++ для применения в бортовых системах космических аппаратов (КА). Сравниваются языки программирования python и c++ с точки зрения производительности и компактности исполняемых решений в условиях деградации элементной базы и ограниченных вычислительных ресурсов. Приводится пример прототипа архитектуры нейросети перцептрон для бинарной классификации рукописных цифр для построения более развитых глубоких моделей.

Ключевые слова: Многослойный перцептрон, нейронная сеть, компилируемый язык программирования C++, Python, космические аппараты, бинарная классификация, распознавание рукописных цифр, производительность, компактность кода.

Введение. Современные космические аппараты (КА) оснащаются бортовыми системами, требующими оперативной обработки больших объемов данных. Нейронные сети (НС) находят широкое применение в таких задачах, как фильтрация и восстановление телеметрической информации как отмечают авторы Гришеленок Д.А., Ковель А.А. [1]. Однако реализация алгоритмов глубокого обучения и прямого распространения в реальном времени на борту космического аппарата (КА) может столкнуться с жесткими ограничениями: деградация электрорадиоизделий (ЭРИ) под воздействием радиации, снижение тактовой частоты и, как следствие, ограниченная вычислительная мощность процессоров, выполненных по большому техпроцессу.

В подобных условиях критически важными становится быстроедействие, а также компактность программного кода. Интерпретируемые языки, такие как Python, предъявляют повышенные требования к аппаратным ресурсам. Как отмечают в своем исследовании Сычёв М. Д., Трунова К. Д. [2], динамическая типизация, наличие сборщика мусора и работа виртуальной машины создают значительные накладные расходы, приводящие к неприемлемым задержкам при обращении к памяти.

Целью данной работы является демонстрация прототипа многослойной нейронной сети (перцептрона), реализованного на компилируемом языке C++.

Предложенная реализация прототипа ориентирована на достижение максимального быстродействия и минимального потребления ресурсов при решении задачи классификации, что делает ее перспективной для использования в бортовой аппаратуре (БА) космического аппарата (КА).

Архитектура прототипа перцептрона. В качестве примера прототипа выбрана задача бинарной классификации рукописных цифр от 0 до 9. Данные представляют собой линейно неразделимое множество, что требует использования многослойной архитектуры для их корректной классификации [3]. Как считает Скобцов «Полносвязные нейронные сети/слои можно рассмотреть на примере следующего 2-слойного MLP, который задан формулами

$$Z^{(1)} = W^{(1)}X + b^{(1)}, \quad A^{(1)} = a_1(Z^{(1)}), \quad Z^{(2)} = W^{(2)}A^{(1)} + b^{(2)}, \quad \hat{y} = A^{(2)} = a_2(Z^{(2)})$$

[4, с. 104], что демонстрирует принцип архитектуры многослойного перцептрона.

Для решения задачи используется многослойный перцептрон. Входной слой составляет 784 нейрона, он получает вектор столбец 28*28 после нормализации исходного изображения, составляющего матрицу 128 * 128 пикселей с бинарным обозначением пикселя. Скрытый слой составляет 128 нейронов с сигмоидальной функцией активации. Выбор данной функции активации обусловлен сложностью $O(n)$ и областью значений от 0 до 1, что делает возможным предсказывать вероятность цифры при бинарной классификации линейно неразделимых данных [5, с. 42]. Выходной слой состоит из 10 нейронов с сигмоидальной функцией активации. Значение на каждом выходном нейроне интерпретируется как вероятность принадлежности входного образа к соответствующей цифре (от 0 до 9).

Программная реализация на C++. Нейросеть реализована в отдельном классе `NeuroNetwork`. Для хранения данных и параметров используются контейнеры `QVector` динамической размерности, что обеспечивает эффективное управление памятью в рамках данной задачи. Объявление основных структур данных выглядит следующим образом:

```
QVector<QVector<double>> w1; // Веса от входа к скрытому слою (784x128)
QVector<QVector<double>> w2; // Веса от скрытого к выходному слою
QVector<double> b1; // Смещения (bias) скрытого слоя (128)
QVector<double> b2; // Смещения выходного слоя (10)
QVector<double> hidden; // Выходные значения скрытого слоя после активации
QVector<double> output; // Выходные значения сети
```

Инициализация весов производится малыми случайными значениями в конструкторе класса, что необходимо для "симметрии" и эффективного обучения.

ПРЯМОЕ РАСПРОСТРАНЕНИЕ. Функция forward служит для последовательного прямого распространения входных значений в выходной слой. Для каждого нейрона скрытого слоя рассчитывается взвешенная сумма входов и смещения, к которой затем применяется функция активации.

```
void NyrolNetwork::forward(const QVector<double> &input) {
```

```
    // Вычисление значений скрытого слоя
```

```
    for (int i = 0; i < HIDDEN_SIZE; i++) {
```

```
        double sum = b1[i];
```

```
        for (int j = 0; j < INPUT_SIZE; j++) {
```

```
            sum += input[j] * w1[j][i];
```

```
        }
```

```
        hidden[i] = sigmoid(sum);
```

```
    }
```

```
    // ... (аналогично для выходного слоя)
```

```
}
```

В качестве функции активации выбрана сигмоида $\sigma(x) = 1 / (1 + e^{\{-x\}})$.

```
double NyrolNetwork::sigmoid(double x) {
```

```
    if (x >= 0) {
```

```
        return 1.0 / (1.0 + exp(-x));
```

```
    } else {
```

```
        double ex = exp(x);
```

```
        return ex / (1.0 + ex);
```

```
    }
```

```
}
```

Данная функция имеет константную вычислительную сложность $O(1)$, однако реализация экспоненты является относительно затратной по времени выполнения.

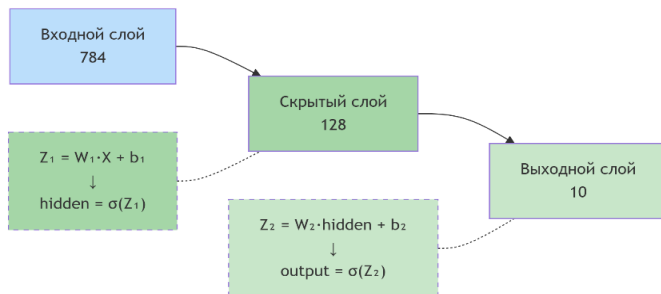


Рис.1. Блок-схема прямого распространения (составлено авторами)

Обучение сети. Обратное распространение. Обучение сети осуществляется с использованием алгоритма обратного распространения ошибки и дельта-правила (градиентного спуска) [3]. Функция `backward` получает на вход вектор входных данных `input`, целевой вектор `target` содержит правильный ответ выходного слоя в соответствии с индексом 0-9 выходного классифицируемого нейрона. Присваивается вероятность 1.0 для остальных 0 и скорость обучения `learningRate`. Для вычисления ошибки, как отмечает Ч. Аггарвал, «Формулу обновления весов по методу градиентного спуска можно переписать в терминах ошибки $E(X) = (y - y_u)$; $w \leftarrow W + aE(x)x$ » [3, с. 33], где a - `learningStep`, а $E(x) \cdot x$ локальный градиент. В случае сигмоиды – это производная. Для функции релу – это константная величина равная 1, что делает ее предпочтительным вариантом по сложности $O(1)$ с соответствующим логическим условием для задач, требующих обработки от 0 до ∞ .

Сначала вычисляются ошибки на выходном слое $E(x)$ и локальный градиент:

```
QVector<double> outputErrors(OUTPUT_SIZE);
for (int i = 0; i < OUTPUT_SIZE; i++) {
    double error = target[i] - output[i];
    outputErrors[i] = error * sigmoidDerivative(output[i]);
}
```

Затем производится корректировка весов w_2 (между скрытым и выходным слоями) и смещений b_2 :

```
for (int i = 0; i < HIDDEN_SIZE; i++) {
    for (int j = 0; j < OUTPUT_SIZE; j++) {
        w2[i][j] += learningRate * outputErrors[j] * hidden[i];
    }
}
for (int i = 0; i < OUTPUT_SIZE; i++) {
    b2[i] += learningRate * outputErrors[i];
}
```

Аналогично, ошибка распространяется на скрытый слой, после чего корректируются веса w_1 и смещения b_1 . Общая вычислительная сложность одной эпохи обучения составляет $O(\text{INPUT_SIZE} * \text{HIDDEN_SIZE} + \text{HIDDEN_SIZE} * \text{OUTPUT_SIZE})$, то есть квадратично зависит от количества нейронов в слоях.

Организация процесса обучения. Для обеспечения сходимости и стохастичности градиентного спуска на каждой эпохе индексы выбранные

цифры обучающих примеров перемешиваются, и веса обновляются после каждого отдельного примера:

```
for (int epoch = 0; epoch < epochs; epoch++) {  
    // ... индексы перемешаны ...  
    for (int idx : indices) { // idx - случайный индекс из перемешанного списка  
        QVector<double> target(OUTPUT_SIZE, 0.0);  
        target[digit] = 1.0;  
        forward(training_data[idx]);  
        backward(training_data[idx], target, learningRate);  
        // ... сбор статистики об ошибке и точности ...  
    }  
}
```

Вычислительная эффективность. Предложенная реализация на C++ обладает преимуществами для встраиваемых систем по сравнению с реализацией на Python. В отличие от Python, который является интерпретируемым языком с динамической типизацией, C++ компилируется непосредственно в машинный код. Это исключает накладные расходы на работу интерпретатора и виртуальной машины. Также за счет прямого доступа к карте памяти (стек, куча) в C++ можно добиться значительной оптимизации.

Для количественной оценки преимущества производительности алгоритмов на языке программирования c++ обратимся к экспериментальным данным. В своем исследовании, посвященном сравнению производительности c++ и python, Сычёв М. Д., Трунова К. Д. провели серию экспериментов по выявлению скорости выполнения алгоритма сортировки пузырьком. Авторы приходят к выводу, что «время выполнения алгоритма на языках C++ и python для 1000 элементов составляет соответственно 0.001с и 0.0292с», и с ростом объема входных данных этот разрыв увеличивается [2]. В частности, на массиве из 20 000 элементов реализация на C++ по сравнению с python продемонстрировала преимущество в скорости выполнения в 25 раз за время 0.496с и 12.4412с. Это наглядно иллюстрирует потенциал отставания Python при выполнении интенсивных математических расчетов. Цикл вычисления взвешенной суммы функции forward, выполняемый миллионы раз со сложностью $O(\text{INPUT_SIZE} * \text{HIDDEN_SIZE} + \text{HIDDEN_SIZE} * \text{OUTPUT_SIZE})$, будет выполняться значительно быстрее в скомпилированной программе. В свою очередь результаты, полученные из опыта сортировки алгоритмом пузырьком, служат убедительной аналогией, демонстрирующей влияние выбора языка на производительность циклических вычислений.

Компактность и предсказуемость. Отсутствие сборщика мусора и возможность точного контроля над выделением памяти [2] делают C++ предсказуемым с точки зрения времени выполнения. Это критически важно для систем реального времени, к которым относятся бортовые системы космических аппаратов (КА). С другой стороны, увеличиваются затраты на г&d разработку и отладку конечного решения. Скомпилированный бинарный код для определенной архитектуры процессора предпочтительней для достижения производительности и оптимизации потребления памяти в условиях деградации ЭРИ.

Демонстрация прототипа. Интерфейс программы позволяет: выбрать датасет, задать количество эпох, задать кол-во примеров на цифру, обучить нейросеть. Рукописный ввод осуществляется на холсте qpainter. После ввода нейросеть предсказывает число.

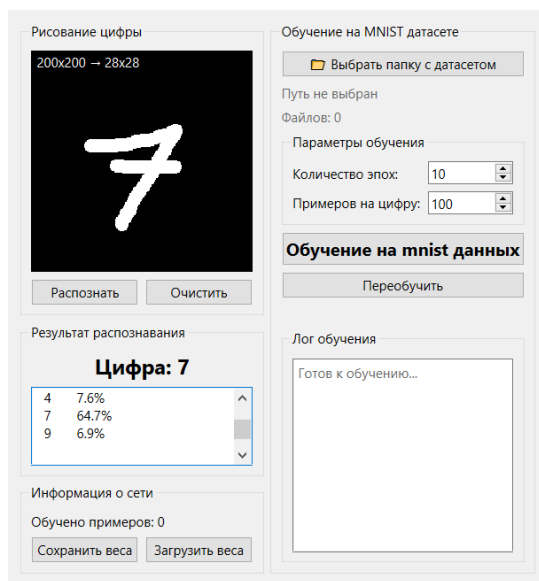


Рис. 2 . Интерфейс прототипа программы

Заключение. В рамках данной работы был разработан и описан прототип многослойного перцептрона на языке C++, предназначенный для классификации рукописных цифр. Основной упор сделан на демонстрацию архитектуры, принципов прямого и обратного распространения, а также на обоснование выбора C++ как языка, обеспечивающего необходимое

быстродействие и компактность для применения в условиях жестких аппаратных ограничений, характерных для бортовой аппаратуры космических аппаратов.

Предложенная реализация подтверждает, что создание эффективных нейросетевых моделей возможно без использования тяжелых фреймворков, что открывает перспективы для более широкого внедрения в специализированные встраиваемые системы с повышенными требованиями к надежности и производительности.

Библиографический список

1. Гришеленок Д.А., Ковель А.А. Использование нейросетей в системах встроенного контроля бортовой аппаратуры космических аппаратов // Исследования наукограда. — 2013. — № 1 (3).
2. Сычёв М. Д., Трунова К. Д. Сравнение производительности при выполнении сортировки методом пузырька на языках программирования C++ и Python // Вестник Пензенского государственного университета. 2024. № 4. С. 131-133.
3. Аггарвал, Ч. Нейронные сети и глубокое обучение / Ч. Аггарвал ; пер. с англ. — М. : Диалектика, 2020. — 752 с.
4. Скобцов В. Ю., Соколов Б. В. Гибридные нейросетевые модели в задаче мультиклассовой классификации данных телеметрической информации малых космических аппаратов // Вестник ВГУ. Серия: Системный анализ и информационные технологии. – 2022. – № 3. – С. 99–114.
5. Митина, О. А. Перцептрон в задачах бинарной классификации / О. А. Митина, П. П. Ломовцев // Национальная ассоциация ученых (НАУ). – 2021. – № 66. – С. 41–46.